SYSTEM DESIGN & VERIFICATION · REVISION 2.0

Distributed Systems & Infrastructure

Three systems built to survive failure: a consensus-replicated store, a non-blocking event pipeline and a GPU fleet control plane. Documented with their designs, their independent audits, and the honest limits of what has been proven.

PHALANX
Raft consensus store
Go · BadgerDB · gRPC

ARGUS
Event ingestion at scale
Java 21 · WebFlux · Kafka

MERIDIAN
GPU fleet control plane
Go · Kubernetes

Basit Tijani
Backend Infrastructure Engineer
github.com/tijani-web

First published May 2026
Revised November 2026 · v2.0
A living document

About this document

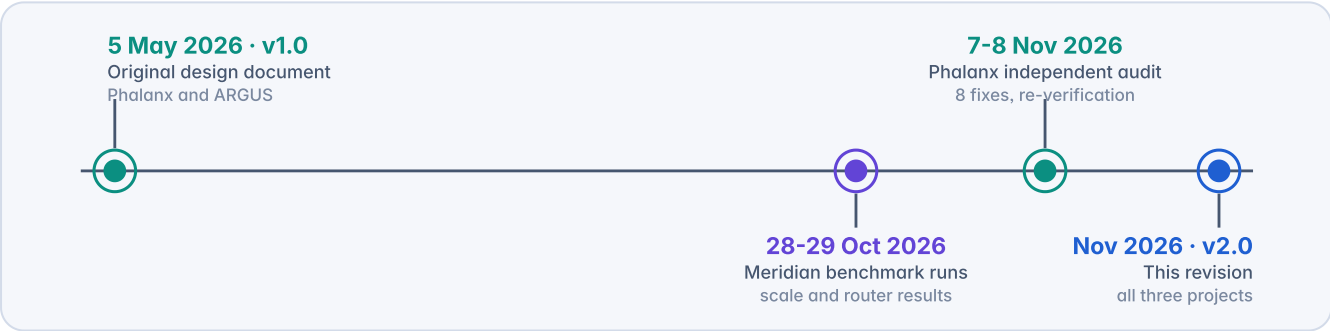
This is a record of how three systems were designed, how they fail, how they were tested, and where the evidence stops. It is written for two audiences at once: a curious newcomer who wants to learn how these systems work, and an experienced engineer who wants to judge the work.

HOW IT IS ORGANIZED

Newcomers: read the shaded *In plain words* and *Concept* boxes. They explain the ideas without assuming a background. **Practitioners:** go straight to the architecture, audit and limitation sections. **Reviewers:** the evidence index near the end points to the raw material behind every claim.

Revision history

REV	DATE	WHAT CHANGED
1.0	May 2026	Original design document: Phalanx and ARGUS architecture, a production incident, and a scale analysis.
2.0	November 2026	Independent audit of Phalanx (8 fixes with commit references and re-verification); Meridian added; ARGUS corrected (rate limiter, network arithmetic, delivery semantics); every claim labeled by evidence level; plain-language guides, concept primers and a glossary added.



Evidence timeline. When each body of evidence was produced. Dates matter: a figure from May and a figure from November do not carry the same weight.

How to read the evidence labels

Every result carries one of three labels. A claim without a label is a design statement, not a result.



The three evidence levels. The labels appear next to results and in the tables throughout.

Contents

- **About this document** 2
- **The short version** 4
- **Engineering principles** 5
- **Phalanx** 6
  - 1.1 Two ideas to know first 6
  - 1.2 The problem and the guarantees 7
  - 1.3 Architecture 8
  - 1.4 Pure state machine design 9
  - 1.5 Concurrency model 9
  - 1.6 The write path 10
  - 1.7 The read path 10
  - 1.8 Storage layout 10
  - 1.9 Elections, membership and snapshots 11
  - 1.10 The independent audit 11
  - 1.11 Performance (local, indicative only) 12
  - 1.12 Cross-region expectations 13
  - 1.13 Incident: cluster bootstrap failure 14
  - 1.14 Design decisions 14
  - 1.15 Known limitations and open questions 15
- **ARGUS** 16
  - 2.1 Two ideas to know first 16
  - 2.2 The problem 16
  - 2.3 Architecture 17
  - 2.4 Reported performance 17
  - 2.5 Three engineering challenges 18
  - 2.6 Invariants and failure intent 19
  - 2.7 Scaling to 10x 20
  - 2.8 Verification status 21
- **Meridian** 22
  - 3.1 Two ideas to know first 22
  - 3.2 What it is 22
  - 3.3 What is real and what is simulated 23
  - 3.4 Key design decisions 24
  - 3.5 Scheduler results (simulated nodes) 25
  - 3.6 Fault tolerance (simulated faults) 26
  - 3.7 Inference router (emulated replicas) 26
  - 3.8 Tested failure modes 27
  - 3.9 Known limitations 27
- **What auditing my own systems taught me** 28
- **Open items** 29
- **Evidence index** 29
- **Glossary** 30
- **Further reading** 31

The short version

If you read one page, read this one.

8
audit findings in my own Phalanx claims, all fixed with tests
VERIFIED

99/100
randomized fault-injection runs checked linearizable
VERIFIED

8-15K
events per second on one 4-vCPU VM (ARGUS)
AUTHOR-REPORTED

0
partially started gangs in simulated fleet runs, vs 1 for default
VERIFIED · SIMULATED

PHALANX
A store that never lies
Five servers on five continents agree on every write. If two fail it keeps working. If three fail it refuses to answer rather than risk a wrong answer.

Solid: an independent audit found eight gaps in my own claims; all were fixed test-first.

Not yet proven: real power-loss durability, wide-area latency, one unexplained failing test seed.

ARGUS
Accept now, store later
An ingestion service that acknowledges each event immediately and does the slow storage work behind a queue, so callers never wait on a database.

Solid: the architecture and three concrete engineering patterns.

Not yet proven: throughput and latency are author-reported; no independent audit yet.

MERIDIAN
All together, or not at all
A Kubernetes add-on that starts GPU jobs all-or-nothing, shares GPUs fairly, saves progress before preempting, and routes inference requests.

Solid: real controllers and scheduler plugins with end-to-end and chaos tests.

Not yet proven: anything on real GPUs; scheduler baselines need strengthening.

One idea runs through all three

Each system is built around a specific way things go wrong: networks split, databases slow down, jobs half-start. The design starts from that failure, then the tests try to cause it. What distinguishes this document is that it also reports where the tests *found something wrong*, because that is the part worth learning from.

Where the evidence stands

PROJECT	EVIDENCE STATUS	LAST VERIFIED
Phalanx	Independently audited, defects fixed test-first, local benchmarks. Wide-area and real-crash testing not done.	7-8 Nov 2026 · commit `49e1a6f` (baseline `1528e85`)
ARGUS	Design documented. Performance figures are author-reported; no independent audit yet.	Figures from May 2026, not re-verified
Meridian	Benchmarks on simulated nodes and an emulated inference server, published with a real-vs-simulated table.	Benchmark runs 28-29 Oct 2026

Engineering principles

Five habits that shaped every decision in the three projects.

01

Failure modes first

Before writing code: what breaks, how does it break, and how long does recovery take? The Raft pure state machine, the atomic Redis rate limiter, the Kafka dead-letter branch and Meridian's atomic gang admission all came from failure analysis, not later optimization.

02

Pure core, impure shell

Keep decision logic free of I/O, clocks and networking where possible. A pure core can be tested deterministically: thousands of consensus rounds run in memory with no waiting.

03

Explicit tradeoffs

Every design decision records what it gains and what it costs. Both Phalanx and Meridian keep decision records. A decision without a cost is usually a decision not yet understood.

04

Measure, then claim

The most valuable thing done on Phalanx was an independent audit of my own documentation. It found claims that were not true of the code. A system documented honestly is more useful than one documented optimistically.

05

Operate what you build

Carrying on-call for the systems you design pushes decisions toward graceful degradation, observability and debuggability at 3 AM.

PART I

Phalanx

A strongly consistent key-value store that keeps working while parts of the world go dark.

Go · Raft consensus · BadgerDB · gRPC · memberlist (SWIM) · 5 regions

Repository github.com/tijani-web/phalanx · Audited at commit `49e1a6f` · Baseline `1528e85` · Audit dates 7-8 November 2026

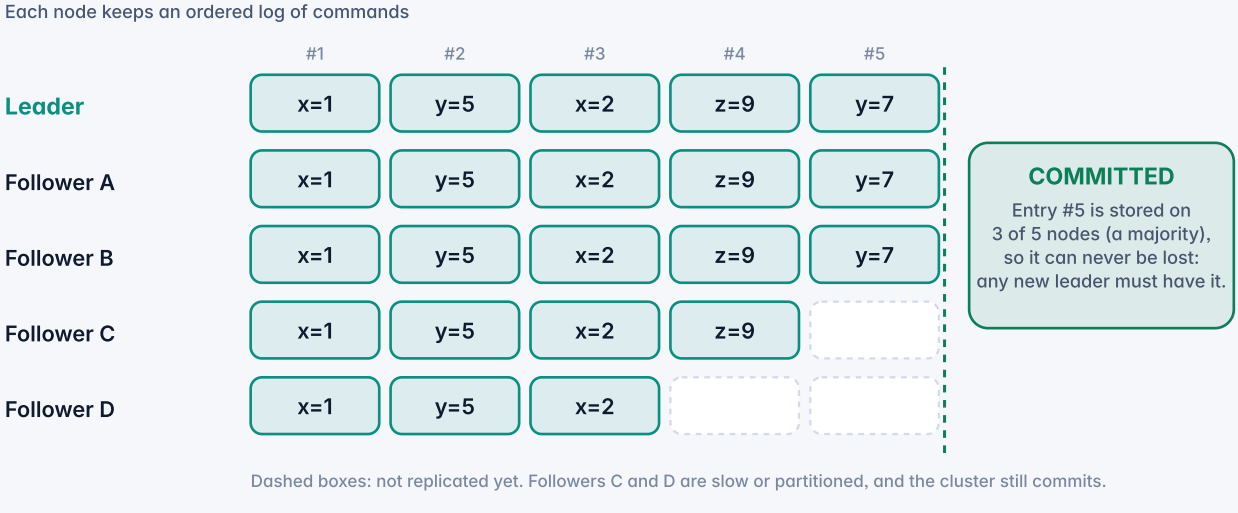
IN PLAIN WORDS

Imagine five librarians on five continents who must always agree on the contents of one notebook, even when the phone lines between them drop. They follow one rule: nothing counts as written until at least **three** of them have written it. Because any three of five overlap with any other three, the group can never end up with two different versions of the truth. That majority rule is called **consensus**, and **Raft** is a way to do it that people can actually understand and implement.

1.1 Two ideas to know first

CONCEPT · RAFT IN 60 SECONDS

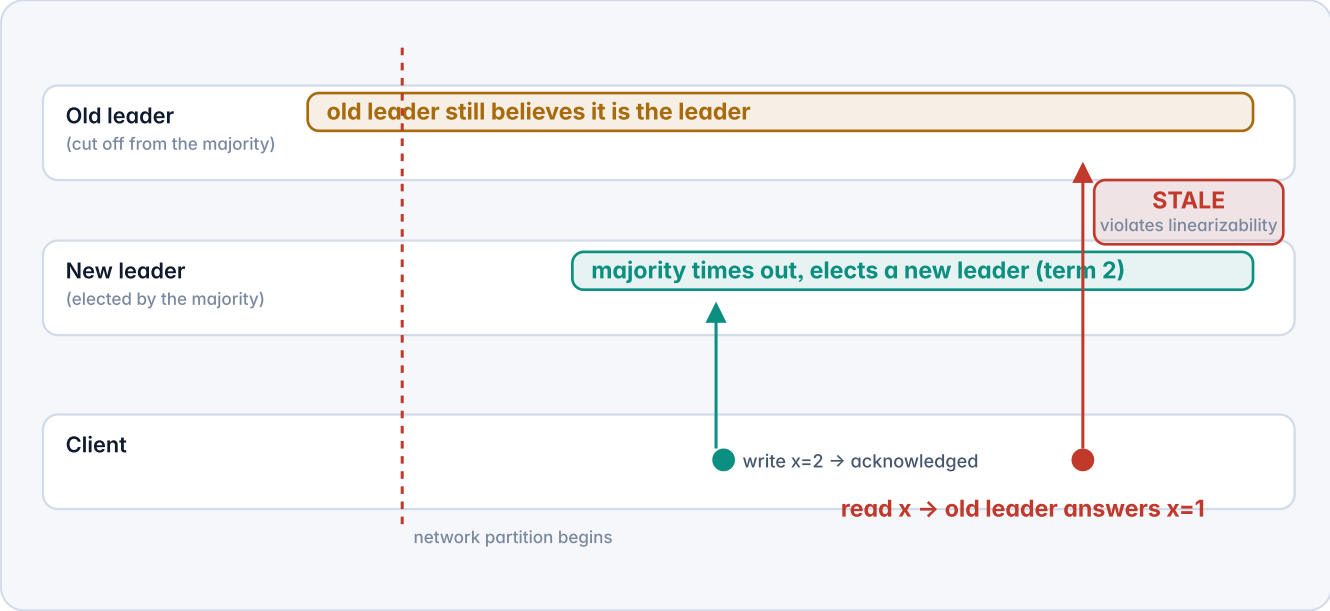
Nodes play one of three roles: **leader**, **follower**, or **candidate**. Time is divided into numbered **terms**, and each term has at most one leader. Clients send commands to the leader, which appends them to its **log** and copies them to the followers. Once a **majority** holds an entry, it is **committed** and applied to the data. If the leader disappears, followers time out, a candidate asks for votes, and whoever gets a majority becomes the new leader. A new leader is only electable if its log is at least as complete as a majority's, so committed entries survive every leadership change.



Log replication. Entry #5 sits on three of five nodes, so it is committed, even though two followers have not caught up. Those two cannot block progress.

CONCEPT · LINEARIZABILITY

A system is **linearizable** if every operation appears to happen at a single instant between when it starts and when it finishes, and everyone sees the same order. Practically: once a write has been acknowledged, *no later read may return the old value*. A server that has been cut off from the majority but does not know it yet is dangerous precisely because it may keep answering reads from stale data. This is the hardest part of a consensus store to get right.



The stale-read hazard. After a partition the majority elects a new leader and accepts writes, while the old leader, unaware, can still answer reads. Phalanx's audit found exactly this failure in the original code (see 1.10).

1.2 The problem and the guarantees

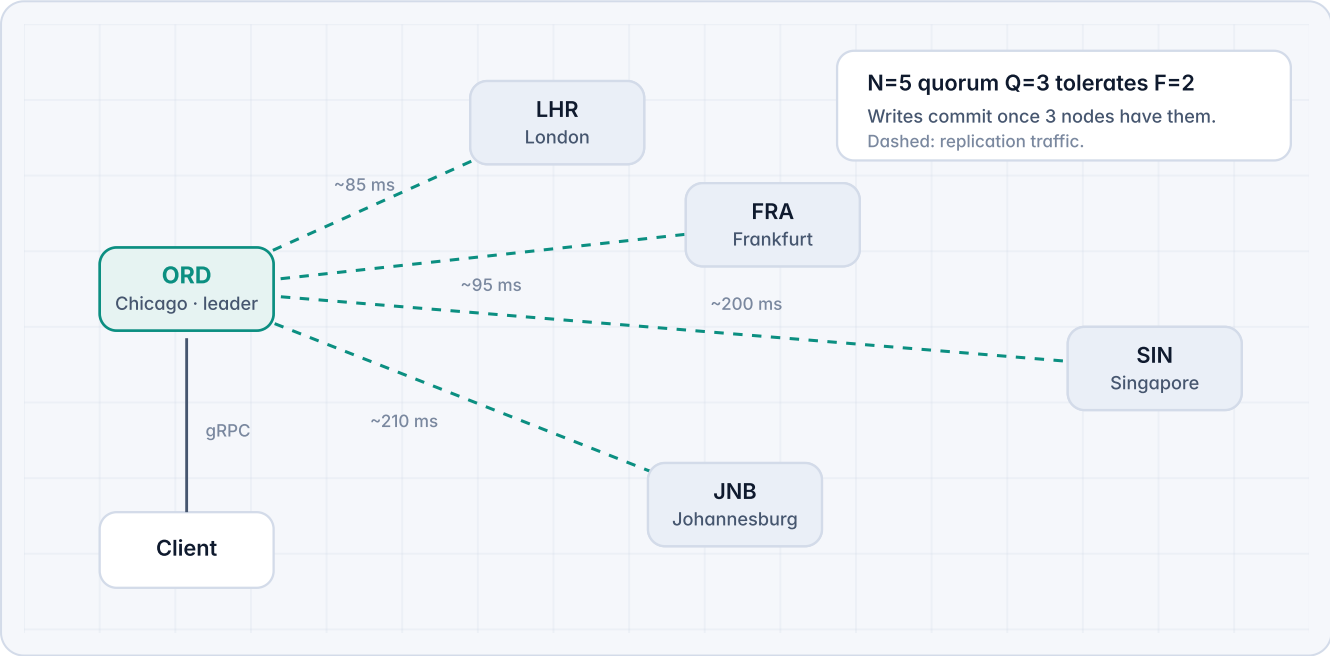
A key-value store spanning regions must stay consistent through network partitions. Leader-and-follower replication cannot promise that; consensus can. Phalanx runs Raft across five nodes: **N=5** nodes, a **quorum Q=3**, tolerating **F=2** failures.

PROPERTY	MECHANISM	STATUS
Linearizable writes	Raft log, majority commit	VERIFIED Porcupine checker: 99 of 100 randomized fault-injection seeds pass; 1 under analysis (1.10)
Linearizable reads	Current-term commit gate plus quorum-contact check	VERIFIED stale-read tests pass. OPEN contact check is tick-based, unproven under process pauses (1.15)
Durability	Log entries fsynced before acknowledgement; log and state replayed on restart	VERIFIED crash-restart and full-cluster restart tests. NOT MEASURED `kill -9` and power loss not tested (graceful stops were used)
Available with 2 nodes down	Quorum 3 of 5	VERIFIED
Refuses service with 3 nodes down	Writes time out; reads return a no-quorum error	VERIFIED
No disruption from rejoining nodes	Pre-vote and leader stickiness	VERIFIED unit tests

PROPERTY	MECHANISM	STATUS
Stale leaders step down	CheckQuorum	VERIFIED unit test
Safe membership changes	Single-server changes through the Raft log	VERIFIED unit tests. OPEN see 1.15
Bounded memory and disk	Snapshots and log compaction	VERIFIED follower catch-up, restart from snapshot. NOT MEASURED long-run memory not measured

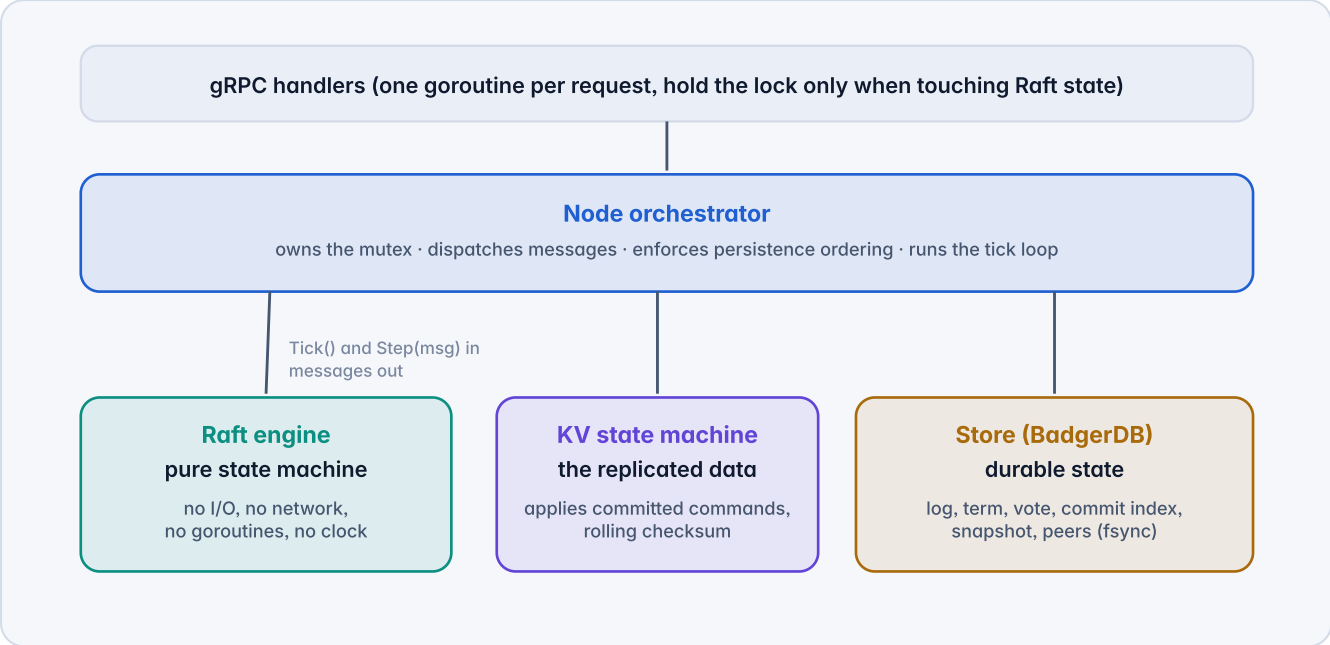
1.3 Architecture

Five nodes, one per region. Peer discovery uses HashiCorp memberlist (SWIM gossip) to *find* candidates; membership itself changes only through the Raft log (1.9). Transport is gRPC over HTTP/2 with a JSON codec. The leader sits wherever the last election placed it; here it is Chicago.



Topology. Round-trip times shown are assumed, approximate values used for reasoning in 1.12, not measurements.

Inside each node there are three layers, and the boundary between them is strict:



One node, three layers. The orchestrator is the only part that touches the network and disk; the Raft engine sees only ticks and messages.

1.4 Pure state machine design

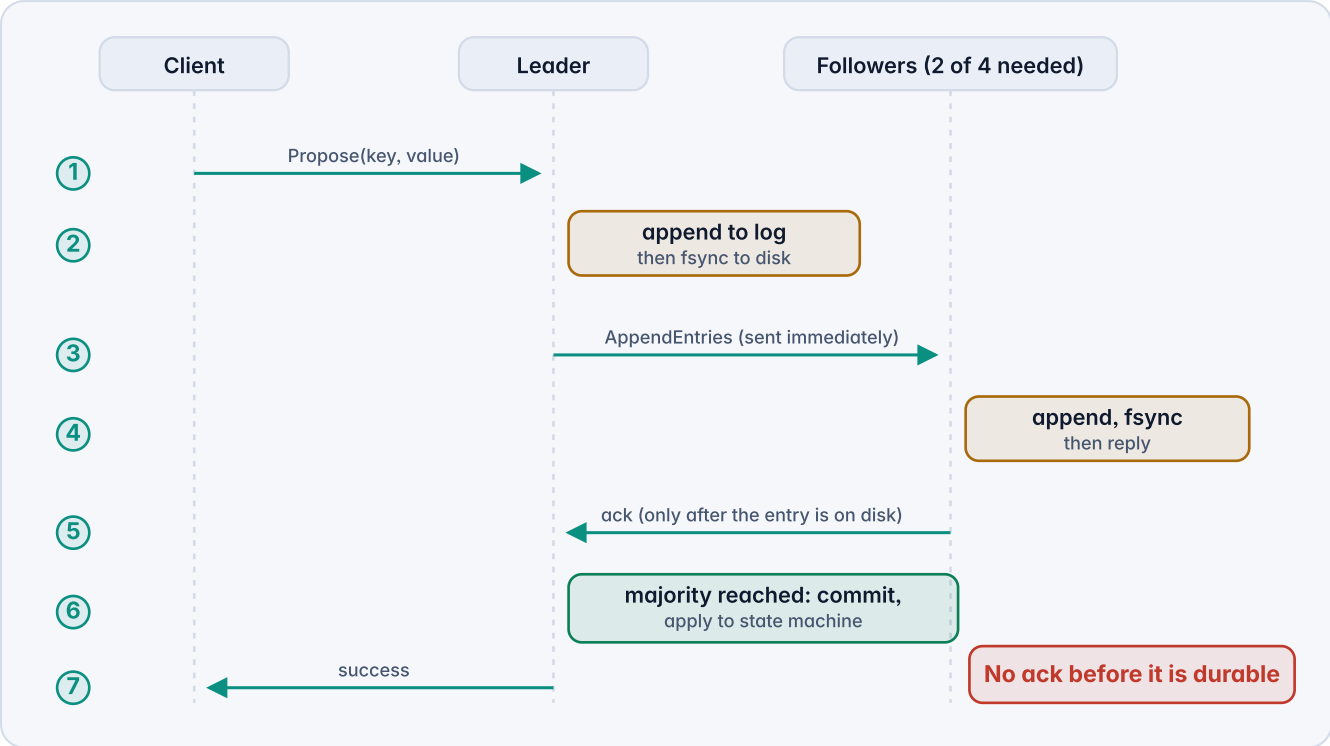
The Raft engine imports no network or storage packages, performs no I/O, and spawns no goroutines. It receives `Tick()` and `Step(msg)` and appends outgoing messages to a buffer. The node layer reads that buffer, persists what must be durable, then dispatches messages.

WHAT THIS GAINS	WHAT IT COSTS
Thousands of consensus rounds run deterministically in memory, with no clock or sleeping	The node layer owns the complexity of dispatch and persistence ordering
No coupling to network, storage or timing; portable to simulators and mocks	Live-network debugging of Raft internals is harder
Persistence ordering rules (1.6) are enforced in one place	A realistic test harness is required to cover timing behavior

1.5 Concurrency model

Multi-threaded I/O, single-threaded consensus. gRPC handlers run as goroutines and take the node mutex only while touching Raft state. A background loop runs `Tick()`, `applyCommitted()`, persistence and message dispatch, serialized under the same lock. Client concurrency scales without ever sharing consensus state across threads.

1.6 The write path



Anatomy of a write. Proposals trigger replication immediately; they do not wait for the next heartbeat tick. Both the leader and each follower persist before they count or acknowledge anything.

> **THE CORE SAFETY RULE**
>
> A follower persists before it acknowledges an append. A leader persists its own entries before dispatching them. A node persists its term and vote before granting or sending a vote request. **Commit latency is bounded below by one round trip to the second-nearest peer**, because the leader plus two followers make a quorum of three.

1.7 The read path

Reads are served only when the leader (a) has committed an entry from its *current* term, which Raft requires before a new leader can trust its commit index, and (b) has confirmed contact with a quorum. This avoids writing a log entry for every read. The quorum-contact check is tick-counted; section 1.15 explains why that is an open question.

1.8 Storage layout

BadgerDB holds everything under prefixed keys: log entries (`l:<8-byte index>`), term, vote, commit index, peer configuration and snapshot. `SyncWrites` is on, so every committed transaction is fsynced. Using BadgerDB's own transactions and crash recovery, instead of a separate write-ahead-log file, keeps a single storage layer. The alternative was an etcd-style WAL or raft-boltdb.

`ValueThreshold` is 64 bytes, which is **lower** than BadgerDB's 1 MB default, so values above 64 bytes go to the value log. Other options are defaults, including the bloom-filter false-positive rate (0.01). Snapshots are stored whole; after compaction, old log keys are deleted and value-log garbage collection runs.

1.9 Elections, membership and snapshots

ELECTIONS

Pre-vote and stickiness

A node that times out first asks peers for *pre-votes* without raising its term. A partitioned node therefore cannot inflate its term and knock out a healthy leader when it rejoins.

LEADER HEALTH

CheckQuorum

A leader that has not heard from a quorum within an election timeout steps down, so an isolated minority leader stops accepting work.

MEMBERSHIP

Changes through the log

Gossip only discovers candidates. Adding or removing a server is a Raft log entry: one pending change at a time, applied on commit and persisted.

SNAPSHOTS

Bounded logs

The log is offset-aware with a sentinel at the compaction point. A follower behind that point receives an `InstallSnapshot`. Snapshots are single messages: fine for KV-sized state, needing chunking for very large state.

Divergence detector. The state machine keeps a rolling CRC64 over applied commands (about 15 ns per operation, AUTHOR-REPORTED micro-benchmark), and followers report it in append responses so the leader can compare. A known limitation is described in 1.15.

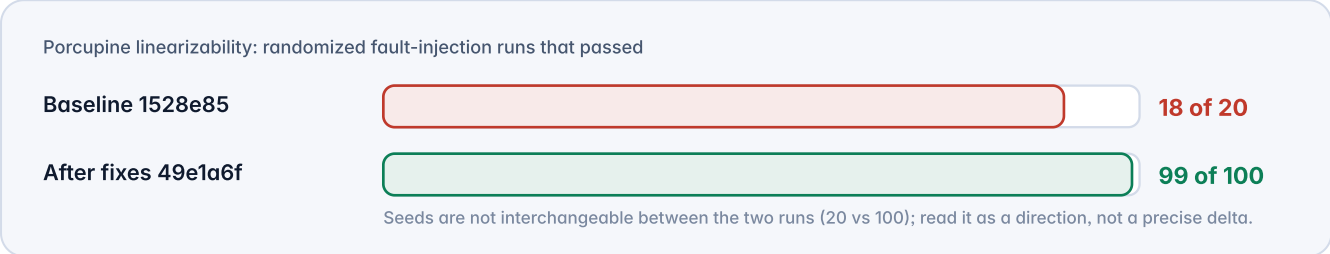
1.10 The independent audit

I ran an audit of the codebase against its own documentation: a code review followed by a correctness suite using the **Porcupine** linearizability checker under partitions, crashes and membership changes. The baseline commit was `1528e85`. It found real defects in claims I had made.

#	BASELINE FINDING	RESOLUTION	COMMIT
1	**Critical.** Raft log entries were never written to disk (memory only, `SyncWrites=false`). A restarted node had an empty log.	Persist and fsync entries before acknowledging; replay on restart	`895ae4e`
2	Reads could be served before the new leader committed an entry from its term; the 'lease' was a heartbeat counter. Porcupine caught stale reads in 2 of 20 seeds.	Current-term commit gate; bounded quorum-contact check	`9b986d8`
3	No CheckQuorum: an isolated leader stayed leader indefinitely.	Step down without quorum contact	`96465c0`
4	Pre-vote was implemented on the receiving side but never initiated.	Initiate pre-vote before any term increase	`af7d706`
5	Gossip join events edited the peer list directly, bypassing consensus.	Bypass removed; changes go through the Raft log	`cb2cc14`
6	No snapshots or log compaction; the log grew without bound.	InstallSnapshot, compaction, value-log GC	`974b584`

#	BASELINE FINDING	RESOLUTION	COMMIT
7	'FSM hash comparison each tick' in my docs did not exist in code.	Rolling CRC64 plus heartbeat comparison	`c1bc799`
8	Bootstrap used a fixed 2-second sleep and proceeded when any one peer resolved.	Quorum-gated start, capped exponential backoff with jitter, background retry	`49e1a6f`

Each fix was written test-first: a test that failed on the old code, then passed. Results after the fixes:



Linearizability under fault injection. Randomized nemesis runs (partitions, leader and follower crashes, membership changes) checked by Porcupine.

- VERIFIED **Durability chaos test:** 30 acknowledged writes survived a full-cluster stop and restart.
- VERIFIED **2 of 5 nodes down:** writes commit and reads succeed. **3 of 5 down:** writes time out and reads return a no-quorum error.

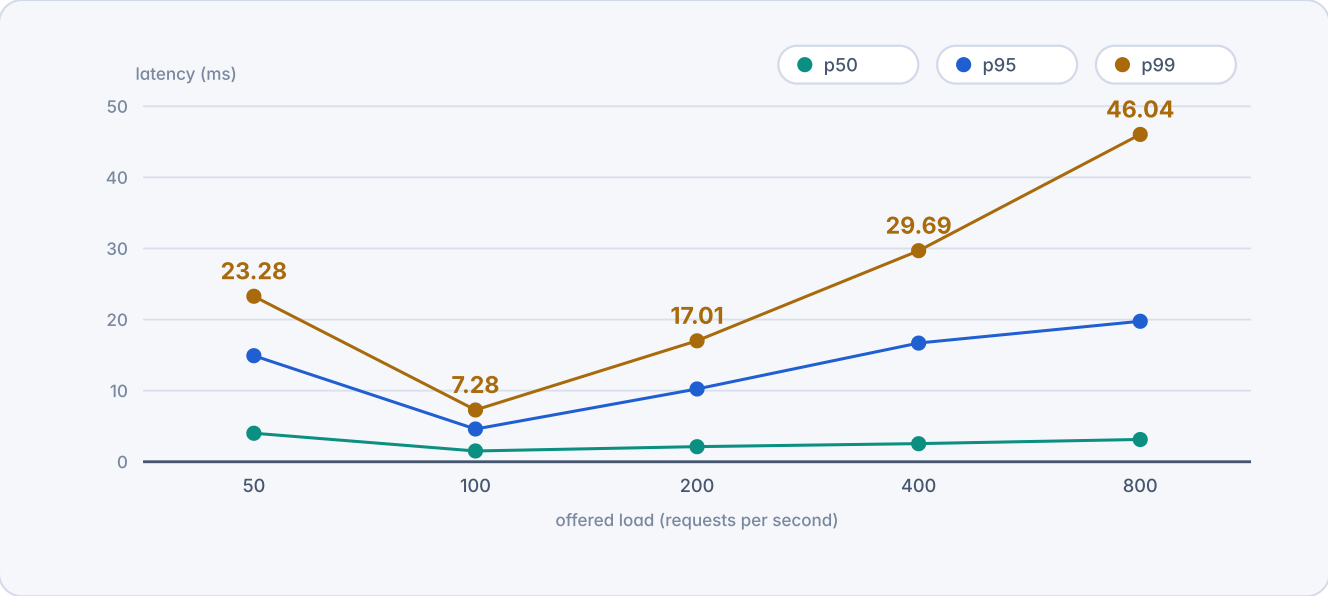
SEED 82 IS UNRESOLVED

A client timed out on a write during a simultaneous leader crash and asymmetric partition, yet the write had in fact committed. The likely cause is how the test harness records timed-out operations: a timed-out write must be modeled as *"may have taken effect at any later time"*, not as failed. That points to the checker model, not a consensus bug, but I have **not** re-run it with that modeling. The failing trace is kept in the repository.

1.11 Performance (local, indicative only)

READ THESE NUMBERS WITH CARE

Measured on a laptop (Intel i7-8665U, 4 cores, 16 GB, NVMe SSD, Windows 11, Go 1.27.1), five nodes as separate processes on loopback, using the **fast test profile** (20 ms ticks). Not production-profile, not cross-region. Runs lasted a few seconds per load level, one repetition, so tail percentiles rest on 100 to 1,500 samples. This is a sanity check, not a benchmark. VERIFIED



Write latency vs offered load (128-byte payload, open-loop arrivals). Completed operations per level: 109, 298, 558, 831 and 1,475.

PAYLOAD @100 REQ/S	P50	P99
64 B	2.20 ms	17.33 ms
1 KB	1.67 ms	9.37 ms
16 KB	12.36 ms	69.74 ms

FAST-PROFILE MEASUREMENT	RESULT
Leader failover (20 trials)	median 376 ms · min 266 · p90 570 · max 597
Follower rejoin (5 trials)	median 47 ms
Cold bootstrap, DNS delay 0 / 2 s / 5 s	257 / 2,305 / 5,610 ms

An earlier baseline run showed worse tail latency under the same harness (p99 of 715 ms at 800 req/s). I did not isolate why, so I make **no causal claim** about the fixes affecting latency.

Storage engine. Three BadgerDB configurations were compared for 5 seconds each on highly compressible synthetic 128-byte values: threshold 64 B async (21.6K ops/s), 1 MB default async (28.3K ops/s, about 4.5x smaller on disk), and 64 B with fsync (25.8K ops/s). That the fsync run beat async shows these runs are within noise of one another. No compaction-heavy workload was run, so **I make no tuning claim** about `ValueThreshold`.

1.12 Cross-region expectations

> **ANALYSIS, NOT MEASUREMENT**
> No real multi-region or emulated-WAN measurement exists yet. What follows is reasoning from an **assumed** approximate round-trip matrix, useful for thinking about where to put the leader.

A commit needs acknowledgements from the leader and two followers, so commit latency is at least the round trip to the **second-nearest** peer, plus fsync time on both ends.

LEADER REGION	TWO NEAREST PEERS (ASSUMED RTT)	COMMIT LATENCY LOWER BOUND
London	Frankfurt 15 ms, Chicago 85 ms	about 85 ms
Frankfurt	London 15 ms, Chicago 95 ms	about 95 ms
Chicago	London 85 ms, Frankfurt 95 ms	about 95 ms

LEADER REGION	TWO NEAREST PEERS (ASSUMED RTT)	COMMIT LATENCY LOWER BOUND
Singapore	London 160 ms, Frankfurt 165 ms	about 165 ms
Johannesburg	London 160 ms, Frankfurt 170 ms	about 170 ms

Consensus latency is dominated by geography, not CPU, so Europe-adjacent leaders are cheapest. The production profile uses 200 ms ticks with a 4-8 second randomized election timeout, so by configuration, failover after a leader loss is on the order of that window. It is a deliberate stability-over-speed choice for high-latency links, and it is tunable.

1.13 Incident: cluster bootstrap failure

A blameless postmortem of the first five-region deployment. AUTHOR-REPORTED

Summary	Nodes failed to form a quorum. The cluster sat in follower state with repeated election timeouts and no leader.
Impact	No cluster available at initial deployment (no user traffic at the time).
Detection	Repeated election timeouts and no leader, with no application-level error. The Raft state machine was behaving correctly.
Root cause	Not a code defect: an infrastructure-timing assumption. Peer discovery used gossip over IPv6 private networks, and nodes resolved peer hostnames immediately at startup. Across five continents, DNS propagation delay meant some nodes resolved a stale or missing record, so peers could not reach each other and quorum never formed.
Resolution	Startup retried DNS resolution instead of resolving once. At the baseline audit this was a fixed 2-second retry (up to 30 attempts) that proceeded when any one record resolved and attempted the seed join only once.
Follow-up	Replaced (commit `49e1a6f`) with quorum-gated startup (proceed once 3 of 5 peers resolve), capped exponential backoff with jitter, and a background task that keeps retrying lagging peers. VERIFIED tested with injected DNS delay.

> **LESSON**
> Distributed systems often fail at infrastructure boundaries (DNS propagation, clock skew, network jitter) rather than in application logic. The architecture must absorb real timing variation. And requiring *all* peers to resolve would itself contradict the fault-tolerance goal: a cluster of five must be able to start with two peers down.

1.14 Design decisions

DECISION	ALTERNATIVE CONSIDERED	WHY
Pure Raft engine, I/O in a node layer	Raft calling storage and network directly	Determinism and testability
BadgerDB for log and state, prefixed keys	Separate WAL file (etcd/wal, raft-boltdb)	One storage layer, existing transactions and recovery
Single-server membership changes	Joint consensus	Simpler state; adequate for N=5
Whole-state snapshot messages	Chunked streaming	Adequate for KV-sized state; revisit for large state

DECISION	ALTERNATIVE CONSIDERED	WHY
Rolling checksum	Full-state hash every tick	A full hash is O(N) per tick (reasoning; not benchmarked)
Long election timeouts in production profile	Aggressive timeouts	Stability on high-latency links

1.15 Known limitations and open questions

AREA	LIMITATION
Crash testing	Restart tests used graceful shutdown. Real `kill -9` and power-loss testing is not done.
Read path	The quorum-contact check is tick-counted. Ticks stop when a process is paused (GC, VM pause, SIGSTOP), so a resumed old leader could briefly believe it still has contact. A wall-clock monotonic lease, or confirming quorum on every read, would close this. The design notes and the audit of the same commit describe this path differently; they should be reconciled.
FSM checksum	Computed over the sequence of applied commands but recomputed from state after a snapshot restore, so values are not comparable between nodes that did and did not restore from a snapshot. An order-independent state checksum would fix this.
Membership	Beyond one pending change at a time, a leader should commit an entry in its current term before accepting a configuration change. New servers join as voters, not learners.
Not implemented	Leader transfer (Raft thesis section 3.10) and joint consensus.
Race detector	`go test -race` needs a Linux or CGO environment. CI is configured, but results were not captured on the benchmark host.
Network	No real multi-region or emulated-WAN measurement.
Seed 82	Unresolved, probably a harness modeling issue (1.10).

WHAT TO TAKE AWAY

- Consensus buys one shared truth through failures; the price is latency bounded by geography, not CPU.
- Bugs hide at boundaries: persistence ordering, read safety, membership. Test the claim, not just the code path.
- Use a checker (Porcupine) instead of intuition, and audit your documentation against your code.

TRY IT YOURSELF

- Open the interactive Raft visualization at raft.github.io, kill the leader, and watch a new term begin.
- For each region, predict the commit latency from its two nearest peers before checking the table in 1.12.
- Record reads and writes from any key-value store and run the history through Porcupine.

PART II

ARGUS

Accept the event, publish it, answer immediately. Never make a caller wait for a database.

Java 21 | Spring WebFlux | Kafka Streams | TimescaleDB | Redis | Azure

Repository github.com/tijani-web/argus · Design and figures from May 2026 · Not yet independently audited

IN PLAIN WORDS

Think of a mail sorting room. Letters (events) arrive at a front desk, get a quick check and a stamp, and go onto a conveyor belt. The desk clerk says "received" at once and never waits for the letters to be filed. Clerks further down the line file them at their own pace. If filing slows down, the desk keeps working and the belt simply gets longer. ARGUS is that arrangement for software events: page views, clicks, latency measurements and health signals arriving thousands of times per second.

2.1 Two ideas to know first

CONCEPT · NON-BLOCKING SERVERS AND THE EVENT LOOP

A reactive server like Netty serves thousands of connections with only a handful of threads. Each thread is like a waiter covering many tables: it must never stand still waiting at one table. If a request makes a **blocking** call (a slow database query), that thread freezes and every other request assigned to it stalls. Reactive programs therefore express work as pipelines that resume when data is ready, and push unavoidable blocking calls onto a separate pool of threads built for waiting.

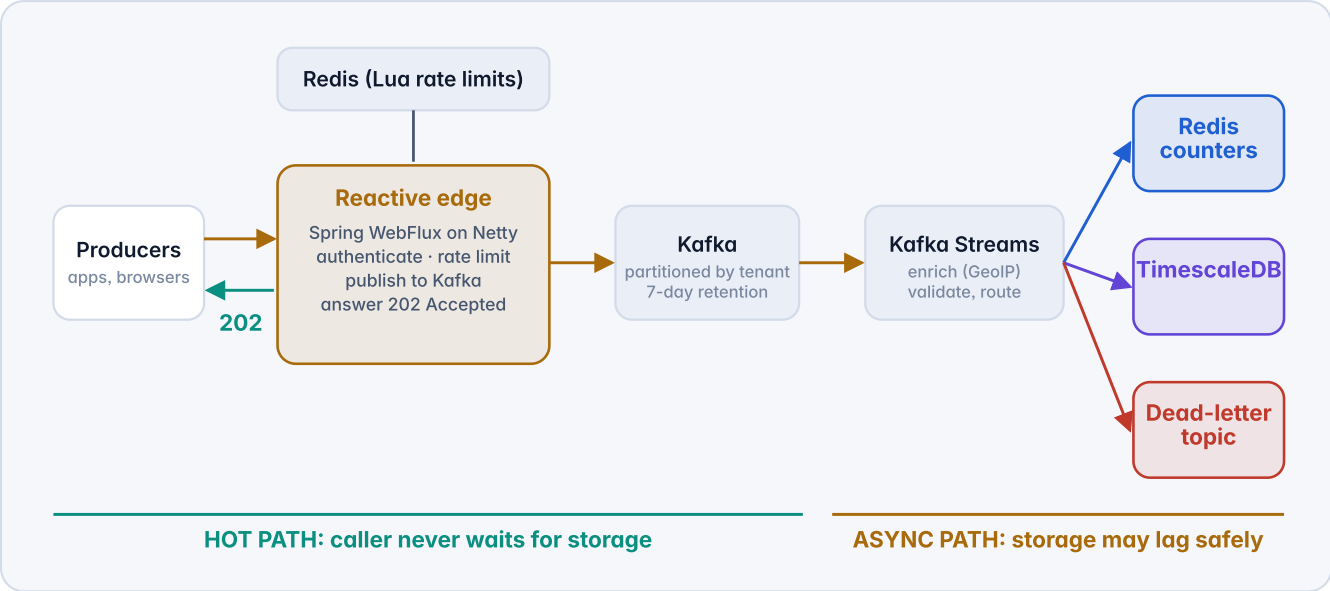
CONCEPT · KAFKA IN 60 SECONDS

Kafka is a durable, ordered log. A **topic** is split into **partitions**; order is guaranteed only *within* one partition, so ARGUS uses the tenant ID as the key to keep each tenant's events in order. Consumers record how far they have read (an **offset**) so they can resume after a crash. Delivery is typically **at-least-once**, so duplicates are possible after retries; the fix is **idempotent** writes, where applying the same event twice has the same effect as once. A **dead-letter queue** (DLQ) is a side topic where unprocessable messages are parked instead of blocking everything behind them. A message that crashes its consumer every time it is read is called a **poison pill**.

2.2 The problem

Applications emit event streams at scale. A naive ingestion pipeline (parse, validate, write to a database) blocks on database latency under load: p99 latency spikes and backpressure cascades upstream to every producer. ARGUS enforces a strict decoupling: **accept the event, publish it to Kafka, return `202 Accepted`. No database write happens on the hot path.**

2.3 Architecture



The ARGUS pipeline. The hot path ends at Kafka; everything after it is asynchronous, so slow storage cannot slow callers.

LAYER	ROLE
Edge (WebFlux on Netty)	Accept events, enforce rate limits, publish to Kafka, return 202 immediately
Kafka	Durable event log; per-tenant ordering through the partition key; the single source of ordering truth
Kafka Streams	Deserialize, enrich with MaxMind GeoIP, route; malformed payloads go to the dead-letter topic
Redis	Minute-granularity per-tenant counters for live dashboards; a TTL bounds staleness
TimescaleDB	Compressed time-series storage partitioned by time, with a 30-day retention policy and efficient range queries

2.4 Reported performance

> **AUTHOR-REPORTED FIGURES**
> The load generator and raw results for these figures are not published here and have not been independently reproduced. See 2.8 for what an audit should confirm.

8-15K
events per second on a single 4-vCPU Azure VM
AUTHOR-REPORTED

8 ms
p50 ingestion latency
AUTHOR-REPORTED

120 ms
p99 ingestion latency
AUTHOR-REPORTED

~3x
throughput gain from the reactive/blocking bridge (2.5)
AUTHOR-REPORTED

METRIC	REPORTED VALUE
Rate limits	50 req/s per IP, 5,000 req/s per API key (atomic Redis Lua)
Storage	about 500 GB for 30 days, compressed

METRIC	REPORTED VALUE
Redis memory	about 2 GB for the 1-hour live-counter window
Effect of the bridge in 2.5	p99 from about 800 ms to about 120 ms during development

2.5 Three engineering challenges

1 · Bridging reactive and blocking code

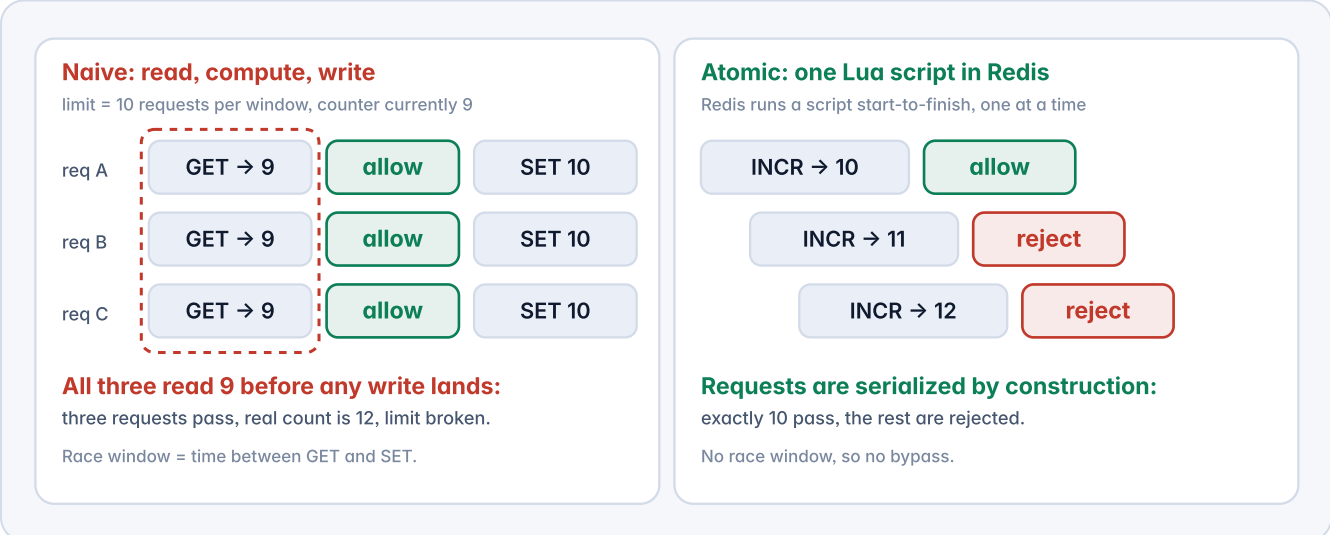
WebFlux runs on a small Netty event-loop pool, and a blocking JPA call on an event-loop thread stalls every other request on that thread. The pattern is to wrap each blocking call and move it to a pool sized for blocking work:

```java
// Blocking repository call moved off the Netty event loop.
// Errors propagate; only "not found" becomes an empty Mono.
public Mono<Tenant> getTenant(String id) {
  return Mono.fromCallable(() -> tenantRepo.findById(id))   // Optional<Tenant>
      .subscribeOn(Schedulers.boundedElastic())
      .flatMap(Mono::justOrEmpty);
}
```

A common mistake is `onErrorResume(e -> Mono.empty())`, which makes a database outage look identical to a missing record. Only genuine absence should become empty.

2 · Atomic rate limiting under concurrency

A naive read-compute-write sequence has a race window: many concurrent requests read the same count before any increment lands, and all of them pass. Moving the whole decision into a single Redis Lua script removes the race, because Redis runs a script from start to finish with nothing interleaved.



Why atomicity matters. The left panel shows three concurrent requests slipping past a limit; the right shows the same requests serialized by one Lua script.

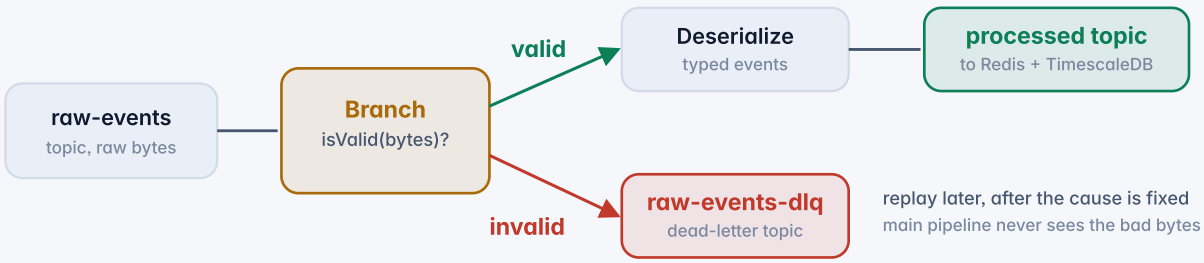
```lua
-- Fixed-window counter. KEYS[1]=bucket, ARGV[1]=limit, ARGV[2]=window in ms
local current = redis.call('INCR', KEYS[1])
if current == 1 then
  redis.call('PEXPIRE', KEYS[1], ARGV[2])   -- set the TTL once, on the first hit
end
if current > tonumber(ARGV[1]) then return 0 end  -- reject
return 1                                          -- allow
```

TWO DETAILS THAT MATTER

The TTL must be set only on the first increment; refreshing it on every request would stretch the window under steady traffic. And this is a **fixed-window** counter, which can permit up to 2x the limit across a window boundary. A token bucket or sliding window is the alternative when that burst matters.

3 · Poison pills in Kafka Streams

By default, one malformed record raises a deserialization exception, which crashes the stream thread; on restart it reads the same record and crashes again. ARGUS reads the topic as raw bytes, branches *before* typed deserialization, and routes failures to a dead-letter topic. The primary topology never sees malformed payloads, and dead-letter events can be replayed once the cause is fixed.



The dead-letter branch. Validation happens on raw bytes, so bad data is isolated before it can crash typed processing.

```java
KStream<String, byte[]> raw = builder.stream("raw-events");
raw.split()
   .branch((k, v) -> isValid(v), Branched.as("valid"))
   .branch((k, v) -> true,       Branched.as("invalid"));
// valid   -> deserialize -> "processed"
// invalid -> "raw-events-dlq"
```

A built-in alternative is Kafka Streams' deserialization exception handler. The explicit branch was chosen for its visible routing and replay path.

2.6 Invariants and failure intent

INVARIANT	MECHANISM
Per-tenant ordering	The partition key is the tenant ID
Completeness: no accepted valid event is dropped	Kafka offsets, at-least-once delivery and idempotent upserts into TimescaleDB. This is *effectively-once through idempotent sinks*, not exactly-once delivery

INVARIANT	MECHANISM
Freshness	A Redis TTL bounds staleness; counters are repopulated from TimescaleDB on a cache miss

Design intent for common failures. DESIGN INTENT These describe how the system is meant to behave, not a record of tested failures.

FAILURE	INTENDED BEHAVIOR
Database slow or down	The hot path is unaffected; the processor lags and recovers from its committed offset
Malformed event	Routed to the dead-letter topic; the pipeline keeps running
Processor crash	Resumes from the committed offset; idempotent upserts make replay safe
Redis loss	Counters are rebuilt from TimescaleDB
One very hot tenant	Concentrates on one partition: the cost of per-tenant ordering

2.7 Scaling to 10x

ANALYSIS At 80,000 to 150,000 events per second:

PRESSURE POINT	ISSUE	RESPONSE
Kafka partitions	8 partitions concentrate tenants; one slow partition becomes a bottleneck	Expand to 32-64 partitions; re-shard by tenant hash
Redis memory	About 2 GB becomes about 20 GB	Shorten TTL to 15 minutes, or shard Redis
TimescaleDB compression	Continuous daily compression competes with queries	Longer chunk interval (1 to 3 days); compress off-peak
Network	At 150,000 events/s of about 1 KB, ingress is about 150 MB/s; with replication factor 3, roughly 450 MB/s across the broker network	Several ingestion nodes behind a load balancer; spread across brokers

The network row assumes 1 KB events and replication factor 3. An earlier version of this document stated 10 GB/s; that was an arithmetic error and has been corrected.

Illustrative monthly cost model per tenant (estimates, not measurements):

RESOURCE	BASELINE: 50K EVENTS/MONTH	SCALED: 500K EVENTS/MONTH
Kafka storage	$20	$200
TimescaleDB	$80	$400
Redis cache	$40	$200
Network egress	$15	$150
Monitoring	$25	$100
Total	**$180**	**$1,050**

2.8 Verification status

NOT YET AUDITED

ARGUS has **not** had the independent audit and re-benchmark that Phalanx had. That audit found eight gaps between what I had written and what the code did, so I do not assume ARGUS is clean. These are the items to verify first.

ITEM TO VERIFY	WHY IT MATTERS
`202` returned only after the Kafka producer ack (`acks=all` , idempotent producer)	Otherwise 'no accepted event is lost' does not hold
Rate-limiter script sets the TTL only on the first increment	A refresh-on-every-request script stretches the window
Repository wrapper does not swallow errors	See challenge 1
Open-loop load test at the stated rates, with published raw output	Turns 2.4 from reported into verified
Dead-letter and replay path under 1%, 10% and 50% malformed traffic	Confirms zero pipeline restarts
Loss and duplicate reconciliation after killing the processor, broker and Redis	Confirms the completeness invariant

Testing approach, as described by the author and not independently audited: Testcontainers (Kafka, TimescaleDB, Redis) for integration tests, stream-branch tests for valid and invalid paths, plus load and chaos tests.

WHAT TO TAKE AWAY

- Decouple acceptance from storage: the hot path should do the minimum and answer.
- Make shared-counter decisions atomic, and give bad data somewhere safe to go instead of crashing on it.
- Be explicit about delivery semantics: at-least-once delivery plus idempotent writes.

TRY IT YOURSELF

- Fire 200 concurrent requests at a GET/SET counter and at an INCR script on a local Redis; count how many pass a limit of 10.
- Feed a malformed record to a Kafka Streams app with no dead-letter branch and watch it restart in a loop.

PART III

Meridian

Teach Kubernetes to schedule GPUs the way large training jobs need it: all together, or not at all.

Go · Kubernetes scheduler framework · controller-runtime · CRDs · KWOK · Helm

Repository github.com/tijani-web/meridian · Benchmark runs 28-29 October 2026 · Simulated nodes, emulated inference

IN PLAIN WORDS

Training a large AI model might need 64 GPUs at the same moment. Getting 63 is worse than useless: those 63 sit idle, costing money, while the job waits for the last one. **Gang scheduling** means the whole group of 64 starts together, or none does. Meridian adds that to Kubernetes, along with fair sharing of GPUs between teams, preemption that saves a job's progress before evicting it, and a router that sends inference requests to the best model replica.

3.1 Two ideas to know first

CONCEPT · GANG SCHEDULING AND THE HALF-STARTED JOB

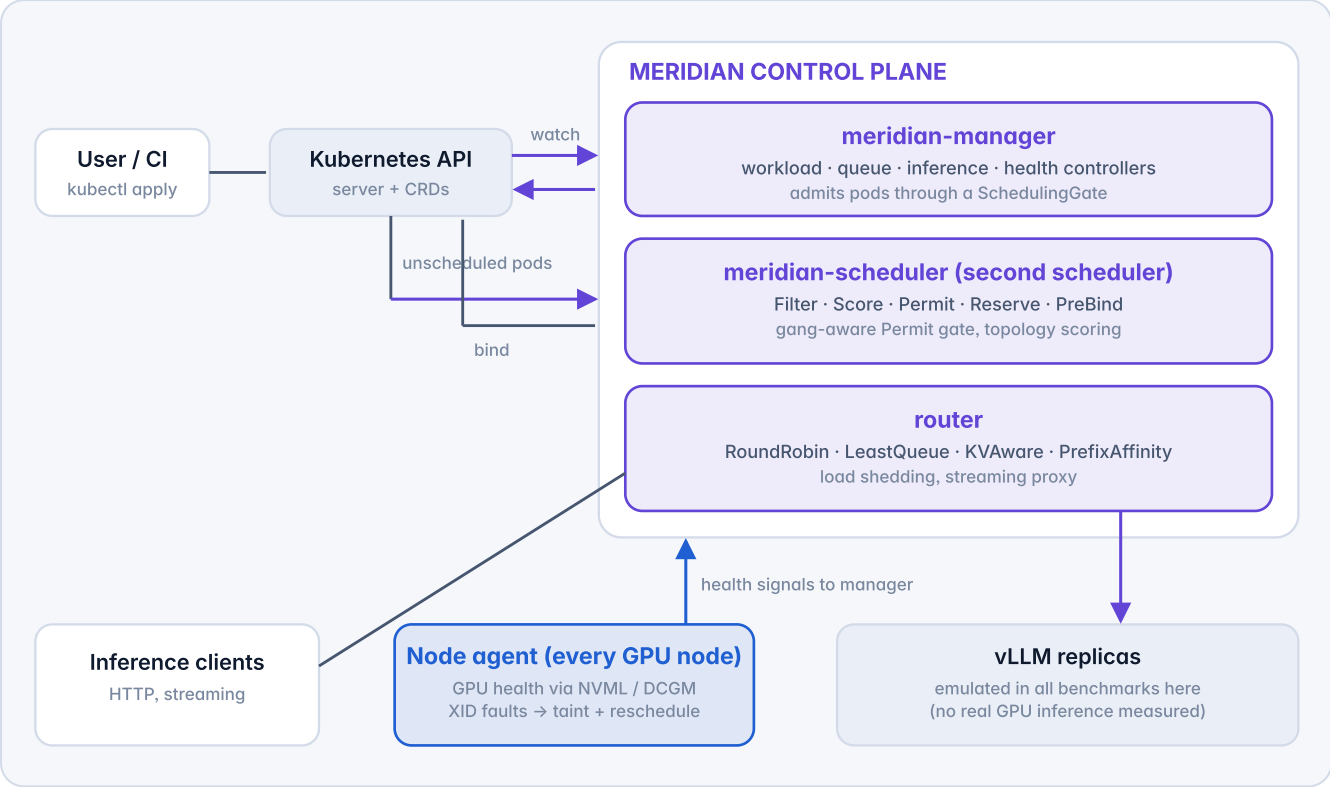
The default Kubernetes scheduler places pods one at a time. For a training job made of many pods that must run together, that is dangerous: some pods can start while others cannot find room, leaving a **partial gang** that holds GPUs but cannot make progress. The wasted resource is measured in **GPU-seconds**. Preventing it needs a decision that is atomic across the whole group.

CONCEPT · HOW THE KUBERNETES SCHEDULER IS EXTENDED

Kubernetes scheduling runs in stages that plugins can hook into: **Filter** (can this node run the pod?), **Score** (which node is best?), **Reserve** (hold the resources), **Permit** (approve, deny or *wait*; this is where a pod can wait for the rest of its gang), and **PreBind** (final preparation before binding). Meridian implements plugins for these stages and runs as a **second scheduler** next to the default one, so ordinary workloads are unaffected.

3.2 What it is

A Kubernetes-native control plane in Go for large-scale LLM training and serving: gang scheduling, checkpoint-aware preemption, fair-share queues, topology placement, canary inference rollouts, and an inference router.



Meridian components. Custom resources (`GPUWorkload` , `GPUQueue` , `InferenceDeployment`) flow through the manager and scheduler; node agents report GPU health; the router fronts the model replicas.

3.3 What is real and what is simulated

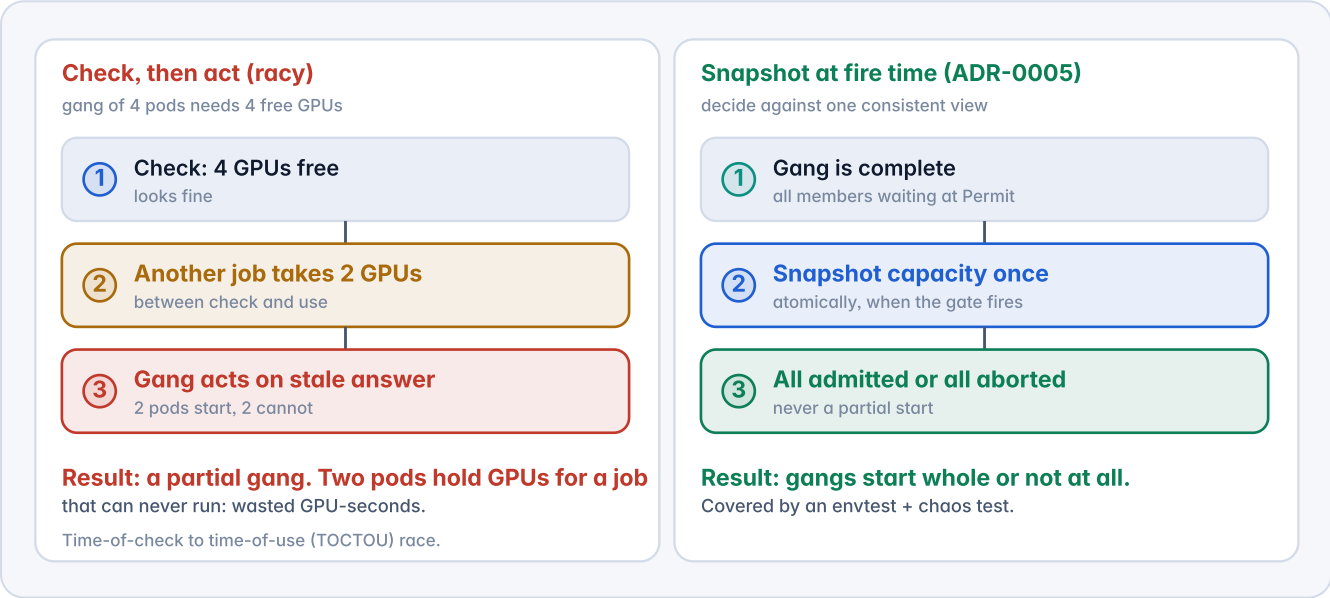
This table is published in the repository and repeated here, because every number in this part depends on it.

COMPONENT	STATUS	NOTES
CRDs, webhooks, controllers	REAL	Full Go implementation; envtest and kind end-to-end tests
Scheduler plugins	REAL	Compiled and deployed as a second scheduler on kind
Checkpoint protocol	REAL	End-to-end tested with an emulated trainer
Inference router	REAL	4 strategies, load shedding, streaming proxy
Node agent	REAL PROCESS	Implemented against the NVML interface. In CI the DCGM-format scrape path runs through a fault injector, because NVML needs a physical GPU
Fleet-scale benchmarks	SIMULATED NODES	KWOK nodes; the Kubernetes scheduling decisions are real
Router latency benchmarks	EMULATED	A vLLM-compatible emulator, not real GPU inference
AWS GPU deployment	UNTESTED	Terraform provided, not validated on real instances

3.4 Key design decisions

DECISION	RECORD
SchedulingGate-based admission instead of watching labels	ADR-0001
Second scheduler instead of replacing the default	ADR-0002
Preemption at the workload layer, not the pod layer	ADR-0003
Extended resources for GPU modeling	ADR-0004
Permit-stage gang abort using a snapshot taken at fire time	ADR-0005

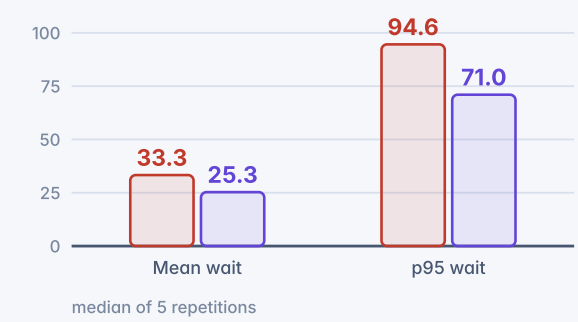
The gang-atomicity design is the most instructive. A naive Permit-stage check can race with other scheduling activity, a **time-of-check to time-of-use** problem: capacity looks sufficient when checked, then changes before the decision takes effect. ADR-0005 documents the snapshot-at-fire approach, covered by an envtest and chaos test.

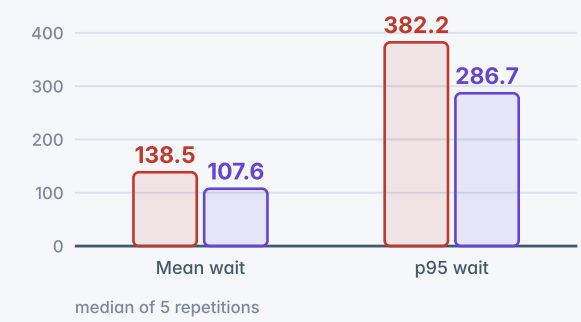


Closing the TOCTOU race. Left: checking and acting at different moments can start half a gang. Right: one atomic snapshot at the moment the gate fires means all-or-nothing.

3.5 Scheduler results (simulated nodes)

VERIFIED Five repetitions per fleet size; medians from `bench/results/2026-10-28-scale-benchmarks/summary.json`. Default scheduler versus Meridian.





500 NODES (4,000 GPUS)	DEFAULT	MERIDIAN
Throughput (pods/s)	344.3	344.3
Mean queue wait (s)	33.3	25.3
p95 queue wait (s)	94.6	71.0
Mean gang start (s)	7.69	7.52
p95 gang start (s)	14.28	14.95
Partially started gangs	1	0
Wasted GPU-seconds	180	0

2,000 NODES (16,000 GPUS)	DEFAULT	MERIDIAN
Throughput (pods/s)	348.1	348.1
Mean queue wait (s)	138.5	107.6
p95 queue wait (s)	382.2	286.7
Mean gang start (s)	31.84	31.96
p95 gang start (s)	60.95	62.07
Partially started gangs	1	0
Wasted GPU-seconds	180	0

WHAT THIS DATA SUPPORTS, AND WHAT IT DOES NOT

- **Supported:** with the default scheduler, which has no gang semantics, a gang can start partially and waste GPU time; Meridian did not in these runs.
- **Throughput is identical** across schedulers and across fleet sizes, including the spread. That points to a bottleneck in the load generator or API server, so it says nothing about scheduler throughput.
- **Allocation was saturated** (peak allocation ratio 1, fragmentation 0), so these runs cannot show placement quality.
- **The queue-wait gain of 22-25%** arrived while gang start time and throughput did not change. The likely source is queue ordering (fair-share), not gang efficiency. Without per-job-size breakdowns, a better mean wait can come at the expense of larger jobs.
- **The partial-gang result rests on very few events,** and the baseline has no gang support. A fairer comparison would include the scheduler-plugins coscheduling plugin, Kueue or Volcano.
- At 2,000 nodes Meridian's gang start is marginally *slower*, within the run-to-run spread.

3.6 Fault tolerance (simulated faults)

50 injected GPU XID faults using a simulated fault injector:

1.24 s
median detection time
SIMULATED

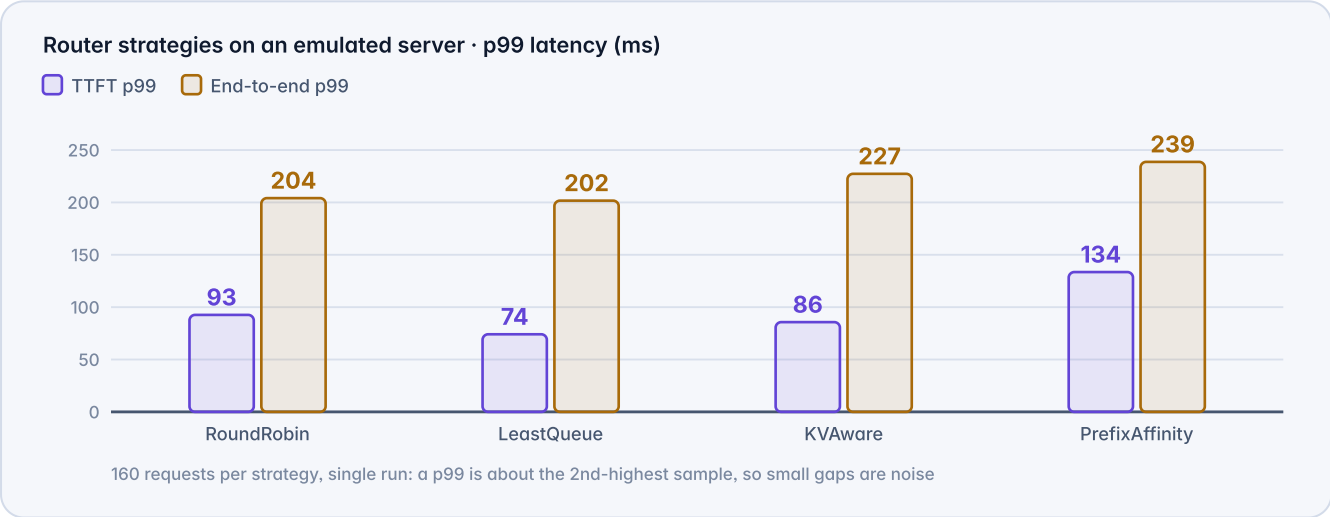
3.46 s
median reschedule latency
SIMULATED

11
steps lost vs last checkpoint
SIMULATED

100%
jobs resumed
SIMULATED

The result file stores one aggregated value per metric, not the 50 individual samples. Detection time is largely set by the scrape interval, and steps lost by the checkpoint interval, so these numbers describe the configuration as much as the system.

3.7 Inference router (emulated replicas)



p99 latency by strategy. 160 requests per strategy, one run, from `bench/results/2026-10-29-inference-router/summary.json`.

STRATEGY	TTFT P50 (MS)	TTFT P99 (MS)	END-TO-END P50	END-TO-END P99	PREFIX HIT RATE	TOKENS/S
RoundRobin	17.6	92.6	92.9	204.1	56.9%	198.2
LeastQueue	20.8	74.1	111.2	201.7	40.6%	190.7
KVAware	16.7	85.7	95.1	227.3	47.5%	189.1
PrefixAffinity	17.4	133.5	102.2	238.7	50.0%	189.6

READING IT HONESTLY

- With 160 samples a p99 is roughly the second-highest observation, so a gap of a few percent (including KVAware's 7.5% TTFT p99 improvement over RoundRobin) is indistinguishable from noise.
- End-to-end latency does not favor KVAware, and RoundRobin had the **highest** prefix-cache hit rate. This workload and emulator cannot separate prefix-aware routing from round-robin.
- Throughput tracks offered load, and shedding was zero in every run, so load shedding was not exercised.
- TTFT from an emulator measures the emulator. The real signal is that all four strategies route and stream correctly. A meaningful comparison needs a prefix-heavy workload, far more requests, and real replicas.

3.8 Tested failure modes

SCENARIO	TEST
Gang partial start (time-of-check/time-of-use race)	envtest and chaos: `test/chaos/gang_atomicity_test.go`
Controller leader failover (target under 30 s)	chaos: `test/chaos/leader_failover_test.go`
GPU XID fault leads to taint and reschedule	kind end-to-end: `test/e2e/node_health_test.go`
Checkpoint, preempt, resume	kind end-to-end: `test/e2e/checkpoint_preemption_test.go`
Canary auto-rollback on a p99 breach	kind end-to-end: `test/e2e/canary_rollback_test.go`

3.9 Known limitations

- No run on real GPUs: NVML paths, real vLLM performance and real KV-cache behavior are untested.
- `GPUWorkload` embeds the full pod template, so the generated CRD schema exceeds the client-side `kubectl apply` annotation limit; use `kubectl apply --server-side`.
- Envtest binaries target Linux and macOS.
- Dynamic Resource Allocation, multi-cluster federation and GPU autoscaling are not implemented.
- Queue quotas are advisory; Meridian trusts its own controllers and is not a hard multi-tenant isolation system.
- Not a general-purpose batch scheduler (not a Volcano or YuniKorn replacement).

WHAT TO TAKE AWAY

- Decisions about a whole group need one atomic snapshot, not check-then-act.
- Label simulation honestly, and ask whether a benchmark measures the system or the harness.
- A comparison is only as convincing as its baselines.

TRY IT YOURSELF

- On a kind cluster with the default scheduler, submit a 4-pod job when only 3 slots are free and watch the partial start.
- Reread the tables in 3.5 and list three extra measurements that would convince you.

What auditing my own systems taught me

LESSON 1

Documentation drifts faster than code

Five claims in my own Phalanx documentation (pre-vote, durability, lease-read safety, state hashing, Raft-managed membership) did not match the code. The fix was a process change: every claim links to a test or result file, and unverified claims are labeled.

LESSON 2

A passing test is not the same as testing the claim

A persistence test existed and passed while the production code path never called it. The audit found this by tracing what the running node actually executes.

LESSON 3

A checker beats intuition

The Porcupine linearizability checker found stale reads that code review had missed. Its one remaining failure was also a lesson: how a client records a timed-out operation is part of the correctness model.

LESSON 4

Benchmarks need a reason to be believed

Identical throughput across two schedulers, tiny samples, and numbers that match a formula are signals that a benchmark is measuring the harness. For every table, ask what could make it look this way for the wrong reason.

LESSON 5

Simulation is fine if you say so

Meridian's real-versus-simulated table is the document I would want from anyone else. Honest labeling costs little and buys a great deal of trust.

LESSON 6

Infrastructure is part of the design

The Phalanx bootstrap incident was not a code bug: DNS timing across continents broke an assumption. Design for the environment, not an ideal one.

Open items

Known, and not currently scheduled.

PHALANX

- Real `kill -9` and power-loss durability tests on Linux
- Wall-clock monotonic lease, or confirm quorum on every read; reconcile design notes and audit
- Order-independent state checksum
- Current-term-commit gate and learner role for membership changes
- Re-run linearizability with indeterminate-operation modeling; 200+ seeds including process pauses
- `go test -race` on Linux CI with published results
- Emulated-WAN and real multi-region measurement
- Long-run memory test showing snapshots bound the heap

ARGUS

- Independent audit of the items in 2.8
- Published open-loop load test with raw output
- Kill-and-reconcile loss test across processor, broker and Redis
- Malformed-traffic test at 1%, 10%, 50%

MERIDIAN

- Gang-aware baselines (coscheduling, Kueue); saturating load; per-queue and per-job-size results
- Fault-tolerance distributions, not single values
- Prefix-heavy, larger router benchmark
- A short session on a real GPU to label some results "real GPU"

Evidence index

PROJECT	WHERE THE EVIDENCE LIVES
Phalanx · baseline audit	`docs/benchmarks/baseline-1528e85/` : audit, findings, discrepancies, benchmarks, raw results
Phalanx · after fixes	`docs/benchmarks/after/` : audit, benchmarks, findings, comparison, raw results; `docs/DESIGN-NOTES.md` ; CI workflow; scripts
Phalanx · correctness	`test/correctness/` : linearizability, lease safety, durability, quorum; plus unit tests per fix
ARGUS	No published benchmark artifacts yet
Meridian	`bench/results/2026-10-28-scale-benchmarks/summary.json` , `bench/results/2026-10-29-inference-router/summary.json` , `docs/LIMITATIONS.md` , `docs/adr/` , `docs/runbooks/` , `test/`

Glossary

AppendEntries
The Raft message a leader sends to copy log entries to followers; also serves as the heartbeat.

Backpressure
Pressure that builds upstream when a downstream stage is slower than the stage feeding it.

BadgerDB
An embedded key-value database written in Go, built on a log-structured merge tree.

CheckQuorum
A Raft safeguard: a leader steps down if it cannot reach a majority.

Commit index
The highest log position known to be stored on a majority; safe to apply.

Consensus
Getting several machines to agree on one value or sequence even when some fail.

CRD
Custom Resource Definition: a way to add new object types to Kubernetes.

Dead-letter queue (DLQ)
A side topic where messages that cannot be processed are parked.

Event loop
A small set of threads that serve many connections by never blocking.

FSM
Finite state machine; here, the replicated data store that applies committed commands.

fsync
A system call that forces data to physical storage, so it survives a crash.

Gang scheduling
Starting all members of a group of tasks together, or none of them.

Hypertable
A TimescaleDB table automatically split into time-based chunks.

Idempotent
Safe to repeat: applying the operation twice equals applying it once.

Linearizability
Every operation appears to take effect instantly, in an order consistent with real time.

Log (Raft)
The ordered list of commands every node must apply identically.

LSM tree
Log-structured merge tree: a storage design that favors fast writes with background compaction.

Nemesis
A test component that injects faults (partitions, crashes) while the system runs.

Offset (Kafka)
A consumer's position in a partition; used to resume after failure.

Open-loop load
Load generated at a fixed arrival rate regardless of how fast the system answers.

Partition (network)
A failure where some nodes cannot talk to others while all remain running.

Porcupine
A tool that checks whether a recorded history of operations is linearizable.

Pre-vote
A Raft step where a node checks it could win before raising its term.

Quorum
A majority of nodes (3 of 5); any two quorums overlap.

Raft term
A numbered election period with at most one leader.

ReadIndex
A Raft technique for linearizable reads by confirming leadership with a quorum.

SchedulingGate
A Kubernetes feature that holds a pod back from scheduling until released.

Snapshot
A saved copy of state that lets the log before it be discarded.

Split brain
Two parts of a system each believing they are in charge.

TOCTOU
Time-of-check to time-of-use: a race where state changes between checking and acting.

TTFT
Time to first token: how long before an inference response starts streaming.

Value log
BadgerDB's separate file for large values, with pointers kept in the LSM tree.

XID error
An NVIDIA GPU fault code reported by the driver.

Further reading

TOPIC	SOURCE
Raft	Ongaro and Ousterhout, *In Search of an Understandable Consensus Algorithm* (2014); visualizations at raft.github.io
Raft in depth	Ongaro, *Consensus: Bridging Theory and Practice*, PhD thesis (2014): membership changes (chapter 4), client interaction and reads (chapter 6), leader transfer (section 3.10), pre-vote (chapter 9)
Correctness	Herlihy and Wing, *Linearizability: A Correctness Condition for Concurrent Objects* (1990)
Testing	Porcupine (github.com/anishathalye/porcupine); Jepsen analyses at jepsen.io
Data systems	Kleppmann, *Designing Data-Intensive Applications*
Kafka and streams	Apache Kafka documentation: Streams, error handling, delivery semantics
Reactive	Spring WebFlux and Project Reactor reference documentation
Time series	TimescaleDB documentation: hypertables, compression, retention
Kubernetes scheduling	Scheduling framework documentation and the scheduler-plugins project; KWOK for simulated nodes
Postmortems	Google SRE book, chapter on postmortem culture

COLOPHON

A living document

Revision 2.0, November 2026. First published May 2026. It will change as the open items close: some claims will be strengthened by new evidence, and some may be corrected. The revision history on page 2 records what changed and when.

Basit Tijani · Backend Infrastructure Engineer

github.com/tijani-web/phalanx

github.com/tijani-web/argus

github.com/tijani-web/meridian

Typeset in Inter and DejaVu Sans Mono. All diagrams are vector graphics drawn for this document.
Evidence labels: Verified · Author-reported · Not measured.